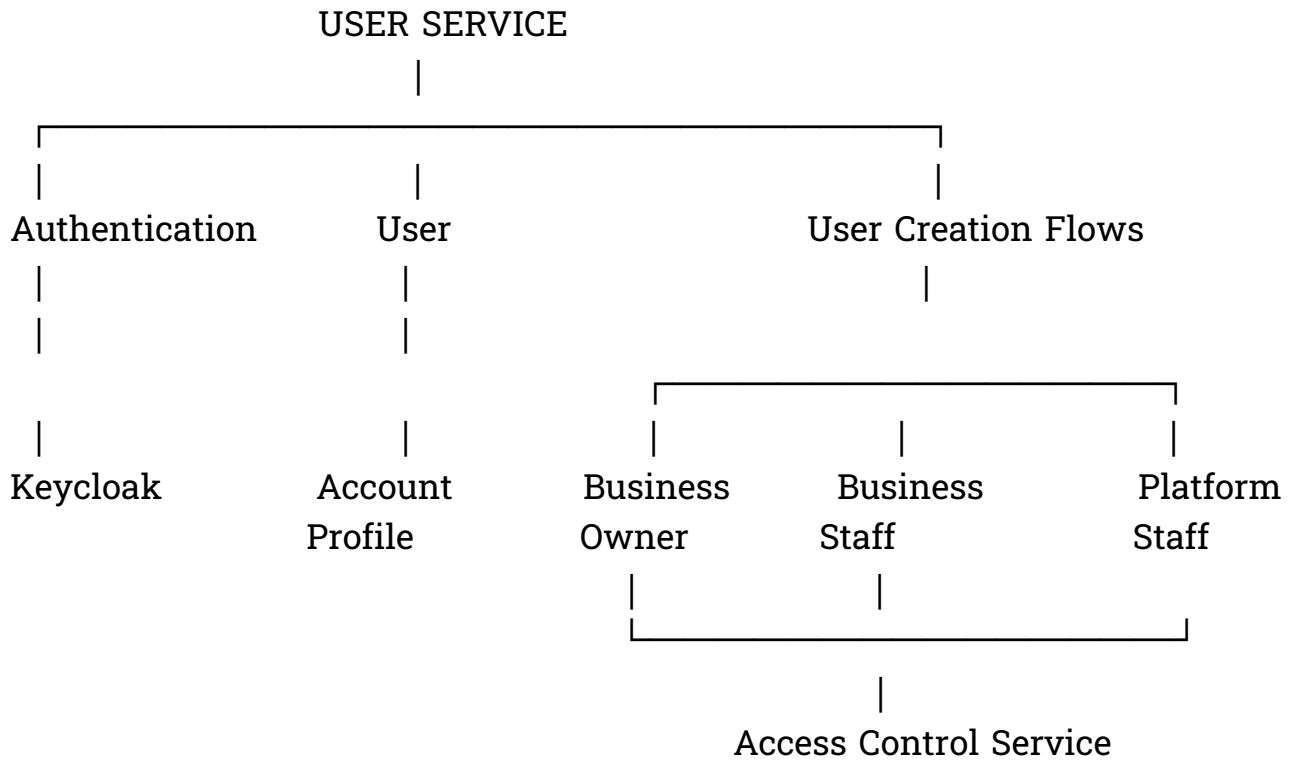


User Service is not just a CRUD service for a user table. It owns user identity integration, user account/profile data, authentication flows, and the creation/orchestration of different types of users.

1. Big picture



There are **five important business domains**:

authentication
user
businessowner
businessstaff
platformstaff

Then:

common
config

are supporting packages, not business domains.

2. user : the core domain

user/

- controller
- dto
- entity
- enums
- mapper
- repository
- service
- impl

This is the **heart of User Service**.

It owns the basic concept:

Who is this person in our platform?

We have two important entities:

UserAccount

UserProfile

UserAccount

Think of it as **system/account information**.

It contains concepts such as:

id

keycloakUserId

username

email

phoneNumber

userType

accountStatus

registrationStatus

It answers:

How does the system identify and manage this user?

UserProfile

This represents more personal/profile information.

Conceptually:

firstName

lastName

gender

photo

...

It answers:

Who is the person behind this account?

This separation is important:

UserAccount = identity/account state

UserProfile = person's profile information

3. UserType is very important

Our current architecture has:

CUSTOMER

BUSINESS_STAFF

PLATFORM_STAFF

SYSTEM_ADMIN

Explanation:

CUSTOMER

Someone using the platform as a customer.

For example:

Customer



Search restaurant

Book restaurant

Book KTV

Manage own profile

BUSINESS_OWNER

Someone who owns a business.

Business Owner

—— Restaurant

—— KTV

Notice that BUSINESS_OWNER does **not** mean:

RESTAURANT_OWNER

KTV_OWNER

as two completely different user types.

Instead, you have:

UserType.BUSINESS_OWNER

plus:

BusinessType.RESTAURANT

BusinessType.KTV

That's an important modeling decision.

4. authentication : authentication use cases

authentication/

—— controller

—— dto

—— keycloak

—— impl

—— service

—— impl

Its responsibility is:

Handle authentication workflows without making the rest of the application understand Keycloak APIs.

The flow is:

POST /auth/login

|

AuthController

|

AuthService

|

—— UserRepository

|

—— UserQueryService

|

—— KeycloakUserClient

|

Keycloak

Our AuthServiceImpl first checks:

```
accountRepository.findByUsernameIgnoreCaseAndUserType(  
    username,  
    request.userType()  
)
```

Then calls:

```
keycloakUserClient.login(  
    username,  
    request.password()  
)
```

Then combines:

Keycloak Token + Platform User => LoginResponse

That is why LoginResponse contains both authentication data and user data.

5. Why have authentication.keycloak?

You have:

```
KeycloakUserClient  
KeycloakUserClientImpl  
KeycloakTokenResponse
```

This creates an **integration boundary**.

Bad architecture would spread Keycloak code everywhere:

UserServiceImpl -> Keycloak REST

AuthServiceImpl -> Keycloak REST
BusinessOwner -> Keycloak REST
PlatformStaff -> Keycloak REST

Our architecture instead says:

Application -> KeycloakUserClient -> Keycloak

So application code depends on: KeycloakUserClient

not on the details of Keycloak's REST API.

This is a good example of **Dependency Inversion**.

6. businessowner : owner registration use case

businessowner/

—— controller

—— dto

—— enums

—— facade

—— impl

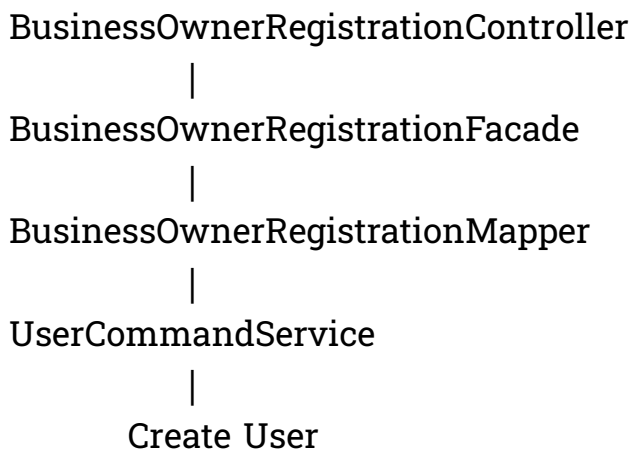
—— mapper

This domain is **not another implementation of user CRUD**.

It represents a business workflow:

Register somebody specifically as a Restaurant/KTV business owner.

The important flow is:



Our facade contains:

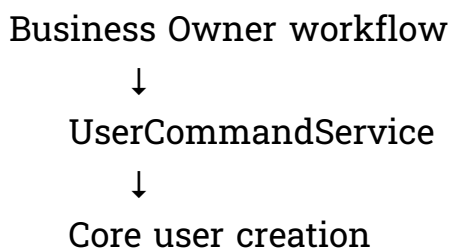
```
return userCommandService.create(  
    mapper.toCreateUserRequest(request)  
);
```

This is an excellent implementation example.

We are **reusing the core user capability**.

We don't create: BusinessOwnerUserService and duplicate all user creation logic.

Instead:



7. Why Facade here but Service in user?

This distinction is worth understanding.

Service

UserCommandService

provides core domain/application capabilities:

create user

update user

delete user

activate user

deactivate user

reset password

Facade

BusinessOwnerRegistrationFacade

coordinates a **larger business use case**.

Today it is simple:

Map request



Create user

Later it might become:

Create User



Create Business Owner Profile



Assign BUSINESS_OWNER role



Create organization



Publish OwnerRegistered event



Send welcome notification

The facade protects the controller from knowing that workflow.

That's a strong enterprise architecture lesson.

8. **businessstaff** : staff belonging to a business

businessstaff/
—— controller
—— dto
—— entity
—— facade
—— mapper
—— repository

This is more interesting because a business staff member is:

User + Business relationship

Our entity: BusinessStaffProfile

contains business-specific information such as:

userId
organizationId
businessId
position
invitedBy
status

Therefore:

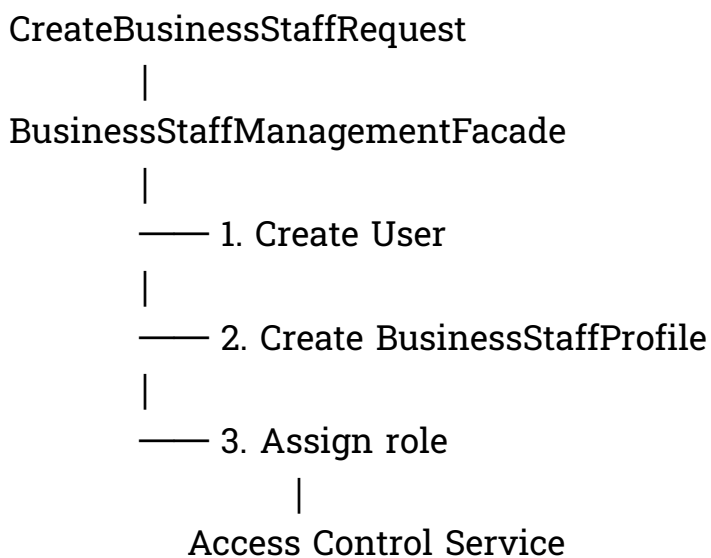
UserAccount -> Who is the user?

BusinessStaffProfile -> What is this user's relationship with this business?

That's a very good separation.

9. Business staff creation flow

Our implementation is essentially:



Our code makes this clear:

```
return userCommandService
    .create(mapper.toCreateUserRequest(request))
    .flatMap(user ->
        createBusinessStaffProfile(user, request)
        .then(assignBusinessStaffRole(user, request))
        .thenReturn(user)
    );
```

This is one of the best methods in this service to demonstrates **orchestration**.

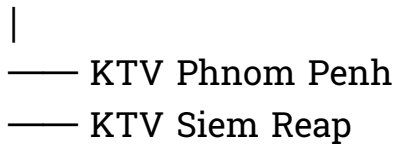
10. organizationId vs businessId

Organization



or:

Organization



So:

organizationId = company/business organization

businessId = particular restaurant/KTV/business unit

Our staff role assignment even supports both scopes:

```
if (request.businessId() != null) {  
    assignDefaultRole(..., request.businessId());  
}  
  
return assignDefaultRole(  
    ...,  
    request.organizationId()  
);
```

Therefore a staff member could conceptually be authorized at Organization scope
or Specific business scope

11. platformstaff : people operating your platform

platformstaff/

—— controller

—— dto

—— facade

—— mapper

This is completely different from business staff.

Business staff

Works for: Restaurant/KTV owner

Platform staff

Works for: The platform itself

For example:

Moderator

Support Staff

Finance Staff

Administrator

Our current facade allows:

PLATFORM_STAFF

SYSTEM_ADMIN

and rejects anything else:

```
if (staffType != UserType.PLATFORM_STAFF
    && staffType != UserType.SYSTEM_ADMIN) {

    return Mono.error(...);
}
```

That's business-rule enforcement.

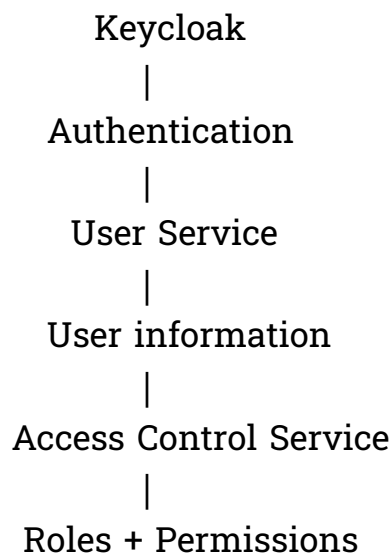
12. Why User Service calls Access Control Service

This is another critical architecture concept.

User Service answers: Who are you?

Access Control Service answers: What are you allowed to do?

Don't mix them.



For example, User Service creates:

```
Sokha
UserType = BUSINESS_STAFF
```

Then Access Control Service gives Sokha:

RESTAURANT_MANAGER

with permissions:

BOOKING_VIEW

BOOKING_MANAGE

MENU_VIEW

MENU_MANAGE

...

That distinction is fundamental.

13. common : shared only inside User Service

common/

—— client

—— exception

—— service

Team may ask:

We already have platform-libraries. Why do we still have common?

Good question.

platform-libraries contains things reusable across **many microservices**.

platform-common

platform-security

platform-web

platform-exception

...

But:

userservice.common

contains things shared by multiple **domains inside this service**.

For example:

UsernameResolver

PasswordPolicy

AccessControlClient

BusinessOwnerClient

These are User Service concepts.

They don't necessarily belong in platform-libraries

The rule is:

Don't put something into platform-libraries just because two classes use it. Put it there only when it is genuinely reusable across services.

14. config

config/

—— AppConfig

—— IntegrationProperties

—— KeycloakProperties

—— PlatformLibrariesConfig

—— SecurityConfig

This is infrastructure configuration.

It answers questions like:

How do we configure Keycloak?

How do we configure external services?

Which endpoints require authentication?

Which platform-library components are enabled?

It should not contain business logic.

15. CQRS in our User domain

You have deliberately separated:

UserCommandService

UserQueryService

This is a lightweight CQRS-style separation.

Commands

Change state:

create()

update()

delete()

activate()

deactivate()

resetPassword()

Queries

Read state:

findById()

findIdentityByKeycloakUserId()

search()

So:

COMMAND

"Do something"

QUERY

"Give me something"

Important point:

CQRS does **not automatically mean Kafka, Event Sourcing, two databases, or microservices.**

Simply separating read and write responsibilities is already applying the core CQRS idea.

16. User creation is the central reusable workflow

UserCommandServiceImpl.create():

```
String username = usernameResolver.resolve(
    request.email(),
    request.countryCode(),
    request.phoneNumber()
);

return passwordPolicy
    .validate(request.password())

    .then(validateUniqueUsername(username))

    .then(createIdentity(request, username))
```

```
.flatMap(account ->
    createProfile(account, request)
        .map(profile ->
            mapper.toResponse(account, profile)
        )
);
```

```
Request
|
Resolve Username
|
Validate Password
|
Check Duplicate
|
Create User in Keycloak
|
Create UserAccount
|
Create UserProfile
|
UserResponse
```

This explains why businessowner, businessstaff, and platformstaff should **reuse UserCommandService** rather than implement user creation themselves.

17. Keycloak vs User Service database

This is probably the single most important concept.

They have different responsibilities.

KEYCLOAK

Password
Authentication
Access token
Refresh token
Credential management
Identity security

USER SERVICE

Platform user ID
Username
Email
Phone
User type
Account status
Registration status
Profile
Business relationships
Application-specific data

They are connected by:

keycloakUserId

Conceptually:

Keycloak User

id = ABC-123

|

| keycloakUserId

|

UserAccount

id = XYZ-789

keycloakUserId = ABC-123

Keycloak owns credentials. User Service owns business user data.

Do not store passwords in your User Service database.

18. One architectural issue I would explicitly discuss

Our current business staff workflow is:

```
@Transactional  
public Mono<UserResponse> createBusinessStaff(...)
```

Inside it we:

```
write PostgreSQL  
  +  
call Keycloak  
  +  
call Access Control Service
```

We might think @Transactional makes all three atomic.

It doesn't.

The database transaction can control our local database operations, but it cannot automatically roll back:

```
Keycloak  
Access Control Service
```

For example:

- | | |
|--------------------------|---|
| 1. Keycloak user created | ✓ |
| 2. UserAccount saved | ✓ |

- 3. BusinessStaffProfile saved ✓
- 4. Access Control call fails ✗

Now we have a partial operation.

This gives us a perfect bridge later in the course to show:

Distributed transactions

Saga

Compensating actions

Outbox Pattern

Event-driven architecture

Idempotency

I would **keep the simple implementation now** rather than immediately complicating it. Then later show students why enterprise distributed systems require more patterns.

19. Another issue worth showing: current search implementation

Our `UserQueryServiceImpl.search()` currently effectively does:

Database



`findAll()`



load users



filter in application



`collectList()`



paginate in memory

This is okay for architecture initially, but it is not what we want at scale.

Eventually it should become:

GET /users?page=2&size=20

|

SQL WHERE

+

LIMIT 20

OFFSET 40

|

PostgreSQL returns only 20 records

That gives us another excellent optimization lesson later.